

VISA

Function Description

Complete the function `lastStoneWeight` in the editor below. The function must return an integer that denotes the weight of the last stone, or 0 if all stones shattered into dust.

`lastStoneWeight` has the following parameter(s):

- `int weights[n]`: an array of integers indicating the weights of each stone

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq weights[i] \leq 10^9$

► Input Format for Custom Testing

▼ Sample Case 0

Sample Input

STDIN	Function
3	→ weights[] size n = 3
2	→ weights = [2, 4, 5]
4	
5	

Sample Output

1

- $1 \leq n \leq 10^5$
- $1 \leq weights[i] \leq 10^9$

► Input Format for Custom Testing

▼ Sample Case 0

Sample Input

STDIN	Function
3	→ weights[] size n = 3
2	→ weights = [2, 4, 5]
4	
5	

Sample Output

1

Explanation

First the worker takes stones with weights 4 and 5 and crashes them into each other. The first one disintegrates completely. The second one is reduced to a weight of 1. Next the worker crashes that stone into the last stone with weight 2. The smaller stone disintegrates, and the last stone is reduced to a weight of 1.

Each day a quarry-worker is given a pile of stones and told to reduce the larger stones into smaller ones. The worker must smash the stones together to reduce them, and is told to always pick up the largest two stones and smash them together. If the stones are of equal weight, they both disintegrate entirely. If one is larger, the smaller one is disintegrated and the larger one is reduced by the weight of the smaller one. Eventually, there is either one stone left that cannot be broken, or all of the stones have been smashed. Determine the weight of the last stone, or return 0 if there is none.

Example

weights = [1,2,3,6,7,7].

The worker always starts with the two largest stones. In this case, the two largest stones have equal weights of 7 so they both disintegrate when smashed. Next the worker smashes weights 3 and 6. The smaller one is destroyed and the larger weighs $6 - 3 = 3$ units. Then, weights 3 and 2 are smashed together, which leaves a stone of weight 1. This is smashed with the last remaining stone of weight 1. There are no stones left, so the remaining stone weight is 0.

Function Description

Complete the function *lastStoneWeight* in the editor below. The function must return an integer that denotes the weight of the last stone, or 0 if all stones

1. Selecting Stocks

An investor has saved some money and wants to invest in the stock market. There are a number of stocks to choose from, and they want to buy at most 1 share in any company. The total invested cannot exceed the funds available. A friend who is a stock market expert has predicted the values of each stock after 1 year. Determine the maximum profit that can be earned at the end of the year assuming the predictions come true.

Example

saving = 250

currentValue = [175, 133, 109, 210, 97]

futureValue = [200, 125, 128, 228, 133]

To maximize profits, the investor should buy stocks at indices 2 and 4 for an investment of $109 + 97 = 206$. At the end of the year the stocks are sold for $128 + 133 = 261$, so total profit is $261 - 206 = 55$.

Function Description

Complete the function *selectStock* in the editor below. The function should return an integer that denotes the maximum profit after one year.

selectStock has the following parameter(s):

int *saving*: amount available for investment

int *currentValue*[*n*]: the current stock values

int *futureValue*[*n*]: the values of the stocks after one year

Sample Case 0

Sample Input For Custom Testing

STDIN	Function
30	→ saving = 30
4	→ currentValue[] size n = 4
1	→ currentValue = [1, 2, 4, 6]
2	
4	
6	
4	→ futureValue[] size n = 4
5	→ futureValue = [5, 3, 5, 6]
3	
5	
6	

Sample Output

6

Explanation

The investor can buy all 4 stocks and gain a profit of $(5-1)+(3-2)+(5-4)+(6-6)$.

► Sample Case 1